

P1083r1 | Move resource_adaptor from Library TS to the C++ WP

Pablo Halpern phalpern@halpernwrightsoftware.com

2018-10-07 | Target audience: Library Evolution

1 Abstract

When the polymorphic allocator infrastructure was moved from the Library Fundamentals TS to the C++17 working draft, `pmr::resource_adaptor` was left behind. It is not clear whether this was an omission or a deliberately conservative move, but the absence of `resource_adaptor` in the standard is a hole that must be plugged for a smooth transition to the ubiquitous use of `polymorphic_allocator`, as proposed in [P0339](#) and [P0987](#). This paper proposes that `pmr::resource_adaptor` be moved from the LFTS and added to the C++20 working draft.

2 Changes from R0

Added a note for LWG to consider clarifying the alignment requirements for `resource_adaptor<A>::do_allocate()`. Changed rebind type from `char` to `byte`. Rebased to July 2018 draft of the C++ WP.

3 Motivation

It is expected that more and more classes, especially those that would not otherwise be templates, will use `pmr::polymorphic_allocator<byte>` to allocate memory. In order to pass an allocator to one of these classes, the allocator must either already be a polymorphic allocator, or must be adapted from a non-polymorphic allocator. The process of adaptation is facilitated by `pmr::resource_adaptor`, which is a simple class template, has been in the LFTS for a long time, and has been fully implemented. It is therefore a low-risk, high-benefit component to add to the C++ WP.

4 Impact on the standard

`pmr::resource_adaptor` is a pure library extension requiring no changes to the core language nor to any existing classes in the standard library.

5 Proposal

This proposal is based on the Library Fundamentals TS v2, [N4617](#) and the July 2018 draft of the C++ WP, [N4762](#).

In section 19.12.1 [mem.res.syn] of the C++ WD, add the following declaration immediately after the declaration of `operator!=(const polymorphic_allocator...)`:

```
// 19.12.x resource adaptor
// The name resource_adaptor_imp is for exposition only.
template <class Allocator> class resource_adaptor_imp;

template <class Allocator>
    using resource_adaptor = resource_adaptor_imp<
        typename allocator_traits<Allocator>::template rebind_alloc<byte>>;
```

Insert between sections 19.12.3 [mem.poly.allocator.class] and 19.12.4 [mem.res.global] of the C++ WP, the whole of section 8.7 [memory.resource.adaptor] from the LFTS v2, including all of its subsections, renumbered appropriately.

LWG NOTE: There was some concern that alignment support was unclear for `resource_adaptor`. We might consider replacing the *Returns* clause of `do_allocate` with something like the following (note the use of normative encouragement in the last sentence:

Returns: a pointer to allocated storage obtained by calling the `allocate` member function on a suitably rebound copy of `m_alloc` such that the size and alignment of the allocated memory meet the requirements for a class derived from `memory_resource`. If the rebound `Allocator` supports over-aligned memory, then `resource_adaptor<Allocator>` should also support over-aligned memory.

6 References

[N4762](#): Working Draft, Standard for Programming Language C++, Richard Smith, editor, 2018-07-07.

[N4617](#): Programming Languages - C++ Extensions for Library Fundamentals, Version 2, 2016-11-28.

[P0339](#): `polymorphic_allocator<>` as a vocabulary type, Pablo Halpern, 2018-04-02.

[P0987](#): `polymorphic_allocator` instead of type-erasure, Pablo Halpern, 2018-04-02.